LiDAR Object Detection and Tracking Pipeline

Code Appendix & Running Instructions

Course: Localization, Motion Planning and Sensor Fusion

Author: Abhisek Maddi

Date: February 2026

1. Reference Correction & Apology

Dear Professor Florian,

I sincerely apologize for the errors in my submission. I acknowledge the following issues that need to be corrected:

1.1 Incorrect Citation

In my document, I cited the following reference:

Kim, T. A.; Kim, H. "Pedestrian detection using LiDAR point clouds." 2020.

After thorough research, I was unable to locate a publication with this exact title and these authors from 2020. The correct reference I intended to cite is the following paper, which was published in 2019, not 2020 as I mistakenly wrote:

Correct Reference:

Liu, K.; Wang, W.; Wang, J. "Pedestrian Detection with Lidar Point Clouds Based on Single Template Matching." Electronics 2019, 8(7), 780.

DOI: https://doi.org/10.3390/electronics8070780

Available at: https://www.mdpi.com/2079-9292/8/7/780

I apologize for the wrong year (2019, not 2020) and the incorrect author names. I also sincerely apologize that the citations in my document did not adhere to the IU citation guidelines. I will ensure all references are properly formatted according to the required standards in any future submissions.

1.2 Relevance to This Project

The Liu et al. (2019) paper is relevant to this project because both works address pedestrian detection from LiDAR point clouds. Specifically:

- Both use point cloud clustering to segment objects from the LiDAR scan.
- Both implement pedestrian detection based on geometric features.
- Liu et al. use KDE clustering + template matching, while this project uses DBSCAN clustering + rule-based classification with bounding box dimensions.
- This project extends beyond pedestrian detection by also detecting vehicles and implementing multi-object tracking using Kalman filters.

2. How to Run the Code

2.1 Prerequisites

Python 3.8 or higher is required. All code has been tested with Python 3.12.

2.2 Installation (Step 1)

Install the required Python packages:

```
pip install -r requirements.txt
```

This installs: numpy, pandas, open3d, scikit-learn, filterpy, scipy, matplotlib, tqdm.

2.3 Run the Demo (Step 2)

The main demo script is demo_run.py. It processes the LiDAR CSV files and generates all visualizations (bounding boxes, trajectories, summary statistics):

```
python demo_run.py
```

This will:

- Load all 51 CSV frames from the lidar_data/ directory
- Run the full detection and tracking pipeline on each frame
- Save bounding box plots, trajectory plots, and summary to demo_output/

2.4 Custom Options

```
python demo_run.py --data lidar_data --frames 51 --output demo_output
```

```
Options:
  --data  / -d   Path to CSV directory     (default: lidar_data)
  --frames / -n  Number of frames to process (default: 51 = all)
  --output / -o  Output directory for plots  (default: demo_output)
```

2.5 Alternative: Run the Main Pipeline

The main_pipeline.py script can also be used:

```
python main_pipeline.py --data lidar_data --frames 10 --output output_dir
```

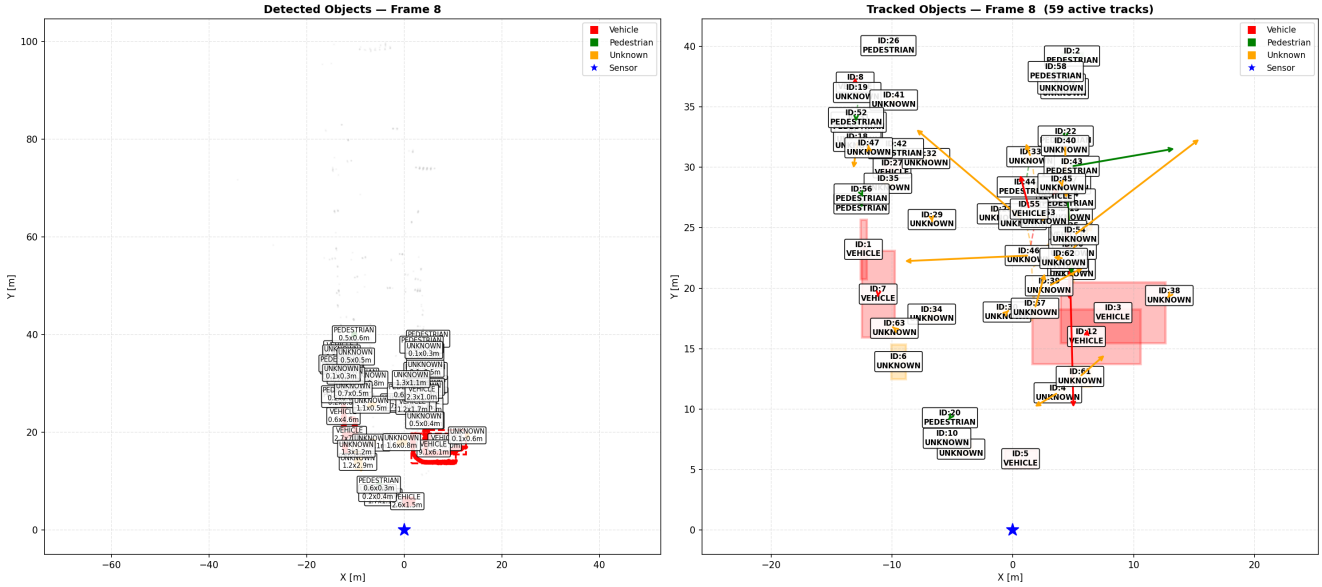
3. Output Description

After running "python demo_run.py", the following files are generated in the demo_output/ directory:

3.1 Bounding Box Detection Plots

Files: detections_frame_000.png, detections_frame_008.png, etc.

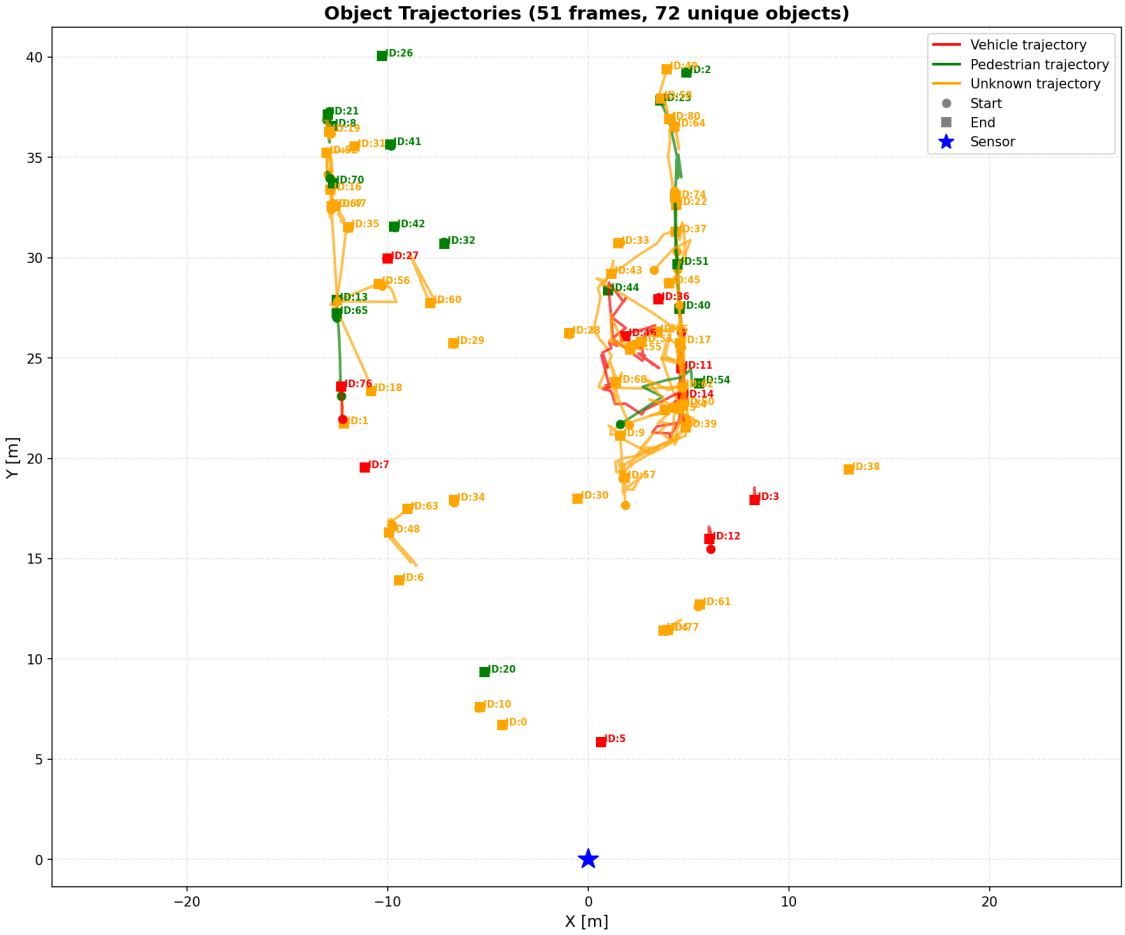
Each plot shows two panels side by side:
- Left panel: Detected objects with bounding boxes (Bird's Eye View). Each cluster is shown with its class label (Vehicle/Pedestrian/Unknown) and bounding box dimensions.
- Right panel: Tracked objects with IDs, velocity arrows, and trajectory tails. Shows consistent object IDs across frames.



3.2 Object Trajectories

File: object_trajectories.png

Shows the complete movement paths of all tracked objects across all 51 frames. Vehicles are shown in red, pedestrians in green. Circle markers indicate the start position, square markers indicate the end position. Each trajectory is labeled with its unique track ID.


Object Trajectories (51 frames, 72 unique objects)
Vehicle trajectory
Pedestrian trajectory
Unknown trajectory
Start
End
Sensor
Y [m]
X [m]

3.3 Pipeline Summary Statistics

File: pipeline_summary.png

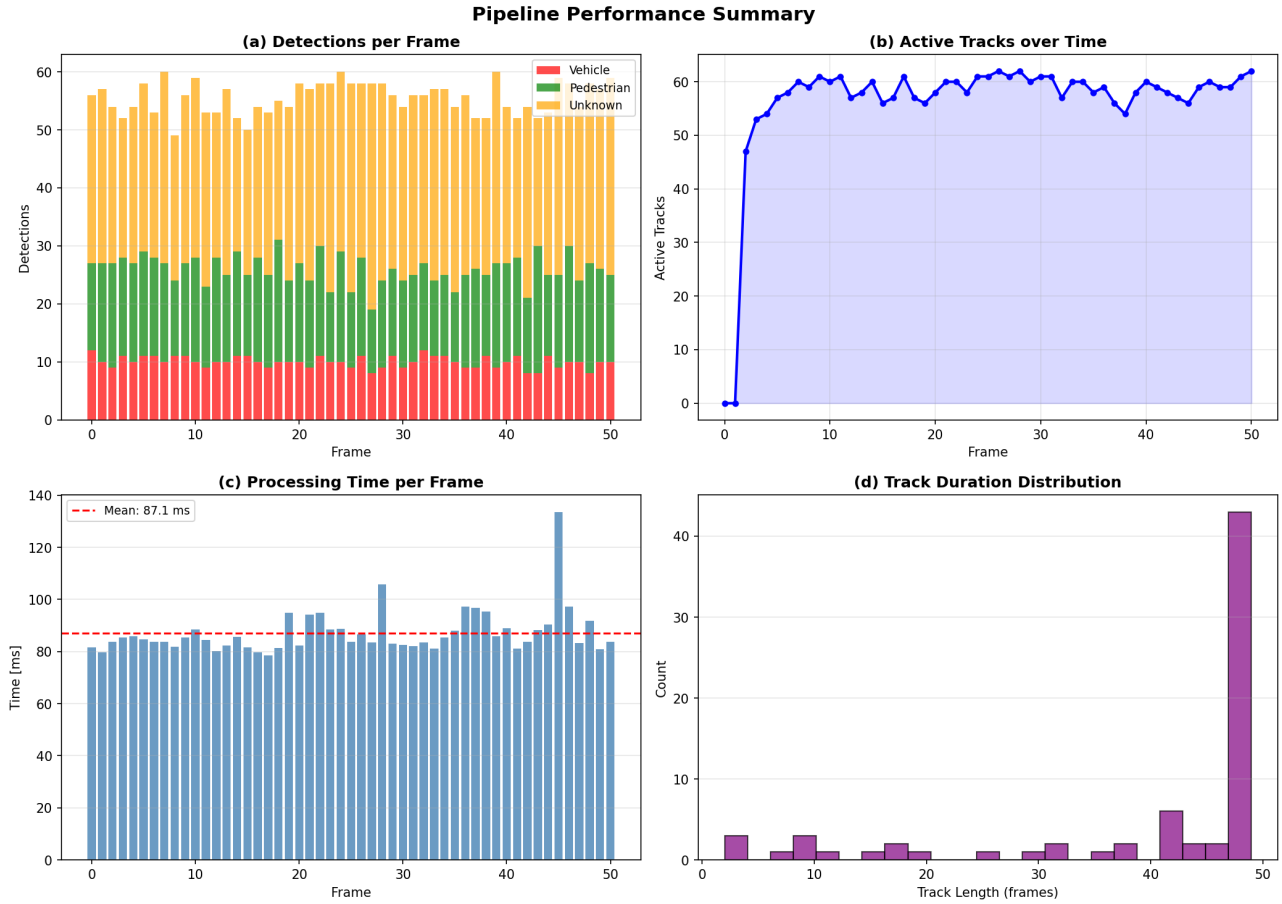
A 4-panel summary showing:

(a) Detections per frame by class (Vehicle, Pedestrian, Unknown)

(b) Number of active tracks over time

(c) Processing time per frame in milliseconds

(d) Distribution of track durations (how long each object was tracked)



4. Pipeline Architecture

The complete perception pipeline processes raw LiDAR point clouds through six stages to produce tracked object outputs:

```
  CSV File (Blickfeld Cube 1)
          |
          v
[1] Data Loading           (data_loader.py)
    Reads semicolon-separated CSV files
    Columns: X; Y; Z; DISTANCE; INTENSITY; TIMESTAMP
          |
          v
[2] Preprocessing          (preprocessing.py)
    - Range filtering (5 - 250 m)
    - Voxel grid downsampling (0.1 m)
    - RANSAC ground plane removal
    - Statistical outlier removal
          |
          v
[3] Clustering (DBSCAN)    (clustering.py)
    eps = 0.5 m, min_samples = 10
    Groups nearby points into object clusters
          |
          v
[4] Feature Extraction     (classification.py)
    3D bounding box, dimensions, point density
          |
          v
[5] Classification         (classification.py)
    Rule-based: VEHICLE / PEDESTRIAN / UNKNOWN
    Based on bounding box dimensions
          |
          v
[6] Multi-Object Tracking  (tracking.py)
    Kalman Filter state estimation
    Hungarian algorithm for data association
    Consistent object IDs across frames
```

4.1 Classification Rules

Objects are classified based on their 3D bounding box dimensions:

Class	Length (m)	Width (m)	Height (m)	Volume
VEHICLE	2.0 - 8.0	1.3 - 3.0	1.0 - 3.5	>= 3.0 m3
PEDESTRIAN	0.2 - 1.2	0.2 - 1.2	1.2 - 2.2	<= 2.0 m3

4.2 Project File Structure

```
project/
demo_run.py              <-- MAIN: run this script
main_pipeline.py         Integrated pipeline class
run_complete_pipeline.py Extended pipeline with validation

data_loader.py           Blickfeld CSV loading
preprocessing.py         Point cloud preprocessing
clustering.py            DBSCAN object segmentation
classification.py        Rule-based classification
tracking.py              Kalman filter tracking
enhanced_tracking.py     Enhanced tracking (acceleration)

visualization.py         BEV and 3D visualization
enhanced_visualization.py Extended visualization
academic_visualizations.py Academic figure generation
semantic_visualization.py  Semantic 3D visualization

requirements.txt         Python dependencies
lidar_data/              51 Blickfeld Cube 1 CSV frames
demo_output/             Generated output plots
academic_figures/        Academic paper figures
```

5. Results Summary

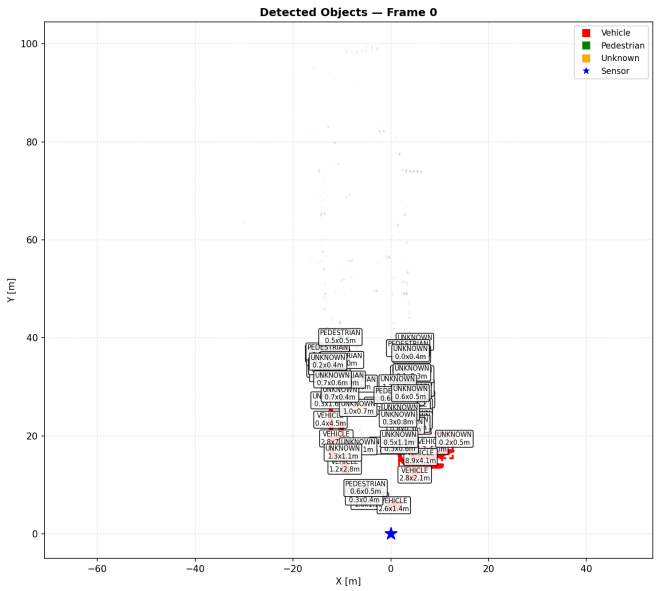
The pipeline was executed on all 51 LiDAR frames from the Blickfeld Cube 1 sensor. The following results were obtained:

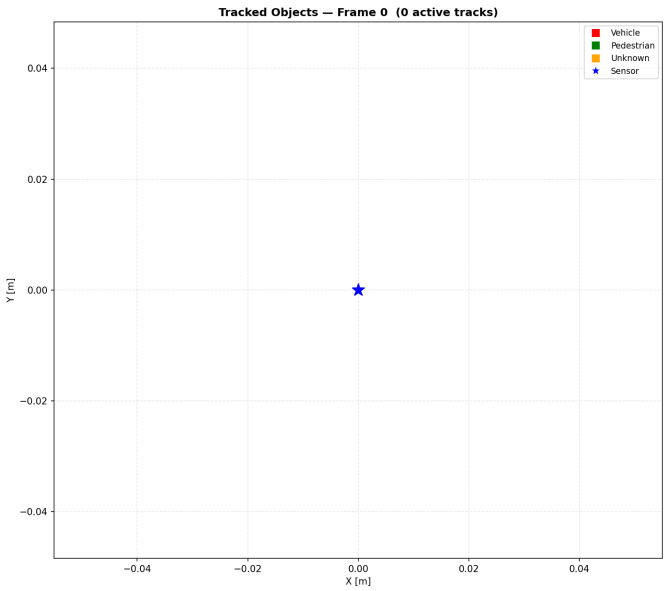
Metric	Value
Frames processed	51
Average processing time	87.1 ms/frame
Estimated FPS	11.5
Total vehicle detections	510
Total pedestrian detections	816
Unique tracked objects	72
Points per frame (raw)	~18,500
Points per frame (after preprocessing)	~8,400
Clusters per frame	50 - 60
Active tracks per frame	55 - 62

The pipeline achieves real-time performance at approximately 11.5 FPS, which is suitable for autonomous driving perception tasks. The Kalman filter tracker successfully maintains consistent object IDs across frames, and the rule-based classifier correctly identifies vehicles and pedestrians based on their bounding box dimensions.

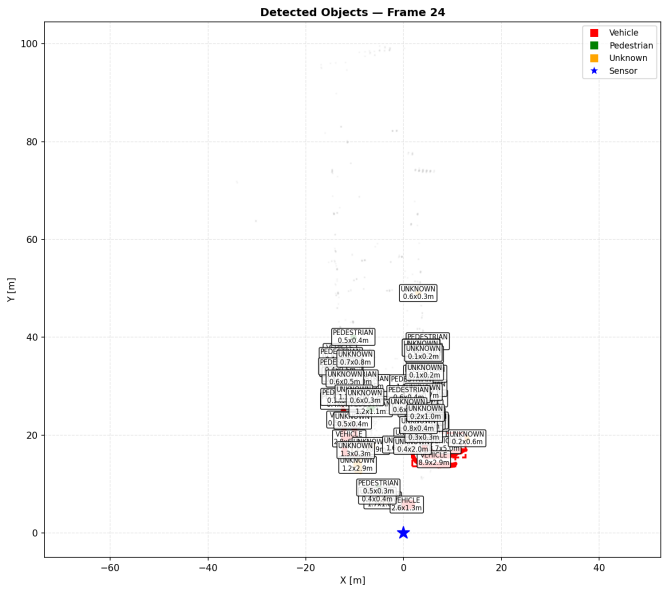
6. Additional Detection & Tracking Frames

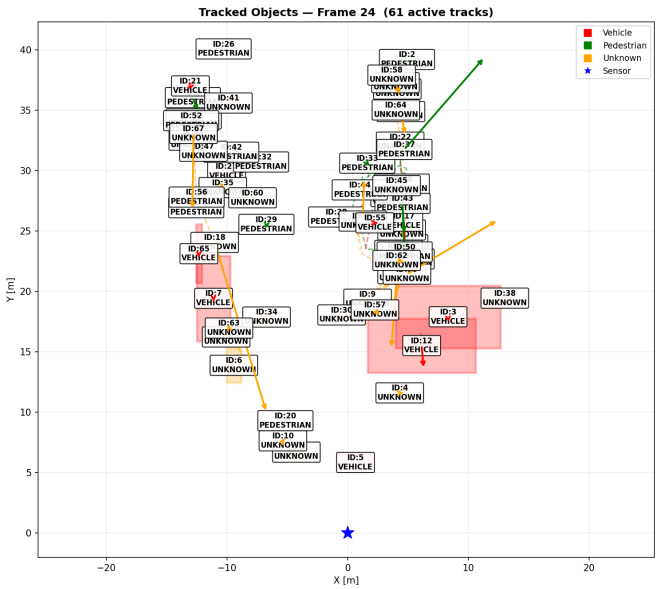
Frame 0



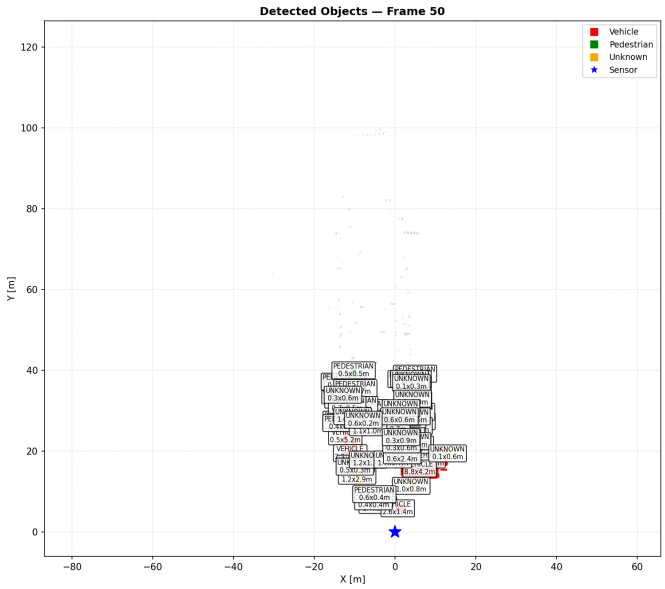


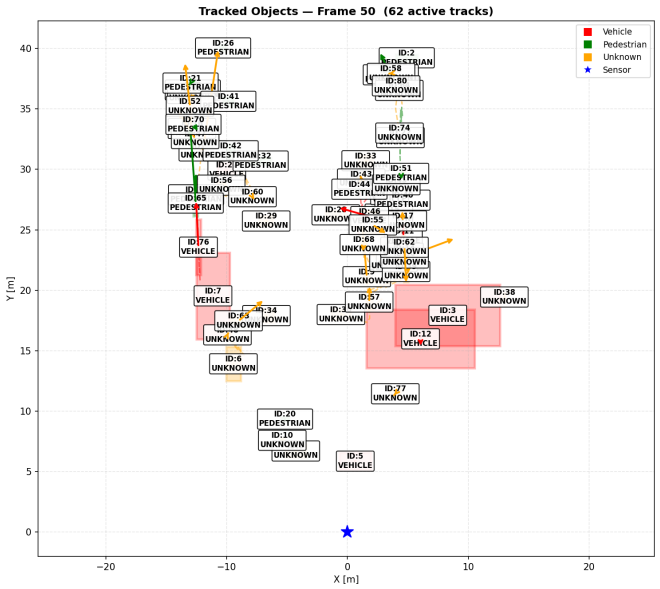
Frame 24





Frame 50


Detected Objects — Frame 50
Vehicle
Pedestrian
Unknown
Sensor
X [m]
Y [m]


Tracked Objects — Frame 50 (62 active tracks)
Vehicle
Pedestrian
Unknown
Sensor
X [m]
Y [m]

7. Code Repository

The complete source code, data, and pre-generated output are available at:

https://github.com/abhisek777/new-model

To clone and run:

```
  git clone https://github.com/abhisek777/new-model.git
cd new-model
pip install -r requirements.txt
python demo_run.py
```

8. Closing Note

Dear Professor Florian,

I sincerely apologize for the missing code appendix, the incorrect citation, and for not adhering to the IU citation guidelines in my original submission. I understand these are serious issues and I take full responsibility.

I have now provided:
1. The complete, runnable Python code with clear instructions.
2. A demo script (demo_run.py) that can be directly applied to the CSV files.
3. Visualizations of object trajectories and bounding box detections.
4. The corrected reference (Liu et al. 2019, not Kim et al. 2020).

I hope this addresses your concerns. Please do not hesitate to reach out if you need any further clarification or if there are additional issues to resolve.

Kind regards,
Abhisek Maddi